Cranfield Search Engine Project

A real-time pipeline with Kafka, Apache Flink SQL and Amazon OpenSearch

Contents

Contents

1. Project overview and architecture 2
2. Data source and exploring the data 2
   Installing the library 3
   Loading the dataset 3
   Reading the documents 3
3. Docker: Dockerfile and Docker Compose 3
   3.1 The Dockerfile 4
   3.2 The Docker Compose file 4
   Understanding the Kafka listeners 5
   3.3 Starting and checking the services 5
4. Kafka producer 6
   4.1 What the producer does 6
   4.2 Libraries 6
   4.3 The .env file 6
   4.4 The complete script (KafkaProducer.py) 6
   4.5 How the key parts work 8
   4.6 Running it 8
5. Flink SQL: validating documents 8
   5.1 Role of Flink 8
   5.2 Technologies 8
   5.3 Opening the Flink SQL client 9
   5.4 The source table 9
   5.5 The valid_documents table 9
   5.6 Validation rules and the INSERT 10
   5.7 The invalid_documents table 11
   5.8 Running both inserts as one job 12
   5.9 Checking that it worked 12
   5.10 How Flink fits into the pipeline 13
6. Kafka consumer: loading OpenSearch 13
   6.1 Introduction 13

6.2 Responsibilities 13
6.3 Libraries and installation 13
6.4 Configuration 14
6.5 The complete script 14
6.6 How it works 17
6.7 Role in the architecture 17
7. Amazon OpenSearch: index, search and results 17
7.1 Creating the domain 17
7.2 The index 17
7.3 The index pattern 18
7.4 Searching the documents 18
7.5 How OpenSearch search works 19
7.6 Results and verification 21
8. Cleaning up AWS resources 21

1. Project overview and architecture

This project builds a small search engine over the **Cranfield collection**, a classic information-retrieval test set of about 1,400 aeronautical research abstracts. Instead of loading the data in one batch, the documents flow through a streaming pipeline:

Stage	Component	What it does
1	Kafka producer (Python)	Reads Cranfield documents and publishes each one as a JSON message to a Kafka topic.
2	Apache Kafka	Buffers the messages and delivers them to whoever reads the topic.
3	Apache Flink SQL	Reads the raw topic, validates each document, and writes it to `valid_docs` or `invalid_docs`.
4	Kafka consumer (Python)	Reads `valid_docs` and indexes every document into OpenSearch.
5	Amazon OpenSearch	Stores the documents in an inverted index and provides full-text search.
6	OpenSearch Dashboards	Lets us run and inspect searches in a browser.

> **Key idea:** Flink and OpenSearch have different jobs. Flink **processes and validates** the stream; OpenSearch **indexes and searches** the result.

2. Data source and exploring the data

The data comes from two places:

- ir_datasets catalogue page for Cranfield: https://ir-datasets.com/cranfield.html
- Cranfield TREC-format copy on GitHub: https://github.com/oussbenk/cranfield-trec-dataset

Installing the library

Install the `ir_datasets` package in your Python environment:

```
pip install ir_datasets
```

Loading the dataset

```
import ir_datasets

dataset = ir_datasets.load("cranfield")
print(dataset)
```

The output is:

```
Dataset(id='cranfield', provides=['docs', 'queries', 'qrels'])
```

What does this mean? `dataset` is an `ir_datasets` Dataset object. It does not hold one table of data; it gives access to three related collections:

Collection	Contents	Used in this project?
`docs`	The research documents: `doc_id`, `title`, `text`, `author`, `bib`.	Yes, these are what we index and search.
`queries`	Example search questions written for evaluating search systems.	No (useful for later testing).
`qrels`	Relevance judgements: which documents are relevant to which query.	No (useful for measuring search quality).

Reading the documents

To look at the documents, loop over `dataset.docs_iter()`:

```
for doc in dataset.docs_iter():
    print(doc)
    break          # show only the first document
```

Behind the scenes, a `for` loop works with an **iterator**:

Concept	Meaning
`iter()`	Creates an **iterator**: an object that lets Python go through a collection one item at a time, without loading everything into memory.
`for` loop	Automatically asks the iterator for the next item, again and again, until there are no items left.

3. Docker: Dockerfile and Docker Compose

We run Kafka and Flink in containers, so there is nothing to install on the machine except Docker. The project uses **both** a Dockerfile and a Docker Compose file, because they solve different problems.

	Dockerfile	Docker Compose
Purpose	**Builds one image**: a recipe that starts from a base image and adds things to it.	**Runs several containers together** as one application.
In this project	Adds the Kafka connector JAR to the Flink image.	Starts Kafka, the Flink JobManager and the Flink TaskManager and connects them.
Command	`docker build` (Compose runs it for you through `build:`)	`docker compose up`

> **Common mix-up:** A Dockerfile does not "pull one image" and Compose does not "pull many images". The Dockerfile **builds** an image; Compose **orchestrates** containers, pulling or building each image it needs.

3.1 The Dockerfile

Flink SQL cannot talk to Kafka out of the box. It needs the **Kafka SQL connector**, a JAR file that must be inside Flink's `lib` folder when Flink starts. The connector version must match the Flink version (`3.3.0-1.20` is built for Flink 1.20).

```
FROM flink:1.20.5

COPY flink-lib/flink-sql-connector-kafka-3.3.0-1.20.jar /opt/flink/lib/
```

Place the JAR in a `flink-lib` folder next to the Dockerfile. The result is a custom Flink image that already contains the connector.

3.2 The Docker Compose file

The Compose file defines three services:

- **kafka**: receives the Cranfield documents and carries messages between the producer, Flink and the consumer.
- **jobmanager**: the Flink coordinator. It accepts jobs, schedules them and serves the Flink web UI.
- **taskmanager**: the Flink worker. It runs the tasks of each job (here with 4 task slots).

```
services:
  kafka:
    image: apache/kafka:4.3.1
    container_name: kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: INTERNAL://:29092,EXTERNAL://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka:29092,EXTERNAL://localhost:9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT,CONTROLLER:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
```

```
    KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"

jobmanager:
  build:
    context: .
    dockerfile: Dockerfile
  image: flink-kafka:1.20.5
  container_name: flink-jobmanager
  ports:
    - "8082:8081"
  command: jobmanager
  environment:
    - |
      FLINK_PROPERTIES=
      jobmanager.rpc.address: jobmanager

taskmanager:
  build:
    context: .
    dockerfile: Dockerfile
  image: flink-kafka:1.20.5
  container_name: flink-taskmanager
  depends_on:
    - jobmanager
  command: taskmanager
  environment:
    - |
      FLINK_PROPERTIES=
      jobmanager.rpc.address: jobmanager
      taskmanager.numberOfTaskSlots: 4
```

Understanding the Kafka listeners

The most confusing part of this file is the three Kafka listeners. Kafka must be reachable from two different places: from other containers and from your own computer. Each place needs a different address.

Listener	Port	Who uses it	Address to use
INTERNAL	29092	Other containers, such as Flink	`kafka:29092`
EXTERNAL	9092	Programs on your computer, such as the Python producer and consumer	`localhost:9092`
CONTROLLER	9093	Kafka's own internal coordination (KRaft mode, no ZooKeeper)	Not used directly

This is why the Flink SQL tables use `kafka:29092`, while the Python scripts use `localhost:9092`. The Flink web UI is published on port **8082** of your computer (`8082:8081` maps host port to container port).

3.3 Starting and checking the services

```
docker compose -f Flink-Kafka-docker-compose.yml up -d --build
docker ps
```

Use the real name of your Compose file. The `--build` flag makes Docker build the custom Flink image from the Dockerfile. `docker ps` should list `kafka`, `flink-jobmanager` and `flink-taskmanager`.

4. Kafka producer

The producer is the **data ingestion component**. It reads the Cranfield documents, converts each one to JSON and publishes it to a Kafka topic, where Flink can consume it.

4.1 What the producer does

1. **Load configuration:** read the broker address and topic name from the `.env` file.
2. **Create the producer:** build a `Producer` from the `confluent_kafka` library.
3. **Load the dataset:** load Cranfield with `ir_datasets`.
4. **Read documents:** iterate with `docs_iter()`.
5. **Prepare the message:** collect `docID`, `title`, `text`, `author` and `bib` into a dictionary.
6. **Serialize:** convert the dictionary to JSON and encode it as UTF-8 bytes.
7. **Publish:** send each message to the topic named in `KAFKA_TOPIC`.
8. **Report delivery:** a callback prints whether Kafka accepted each message.
9. **Flush:** wait until all queued messages are delivered before the program ends.

4.2 Libraries

Library	Purpose
`confluent-kafka`	Connects to Kafka and publishes messages with the Producer API.
`ir-datasets`	Loads the Cranfield dataset.
`python-dotenv`	Loads variables from the `.env` file.
`json`, `os`	Built into Python: JSON conversion and reading environment variables.

```
pip install confluent-kafka ir-datasets python-dotenv
```

4.3 The .env file

```
PRODUCER_CONFIG=localhost:9092
KAFKA_TOPIC=cranfield_docs
```

- `PRODUCER_CONFIG` holds the Kafka bootstrap server. Because the producer runs on your computer, use the EXTERNAL address `localhost:9092`.
- `KAFKA_TOPIC` is the topic that receives the raw documents. The Flink source table in Section 5 must read this same topic. `cranfield_docs` is an example name; use your own.

4.4 The complete script (KafkaProducer.py)

```
from confluent_kafka import *

import os

import json

import time

import ir_datasets

from dotenv import load_dotenv
```

```
load_dotenv()


prod_config = {
    "bootstrap.servers": os.getenv("PRODUCER_CONFIG")
}


prod = Producer(prod_config)


def delivery_report(err, msg):
    if err is not None:
        print("Delivery failed:", err)
    else:
        print(
            f"Message delivered to {msg.topic()} "
            f"[partition {msg.partition()}]"
        )


dataset = ir_datasets.load("cranfield")


for count, doc in enumerate(dataset.docs_iter()):




    doc_data = {
        "docID": doc.doc_id,
        "title": doc.title,
        "text": doc.text,
        "author": doc.author,
        "bib": doc.bib
    }


    value = json.dumps(doc_data).encode("utf-8")


    prod.produce(
        os.getenv("KAFKA_TOPIC"),
        value=value,
```

```
        callback=delivery_report
    )


    prod.flush()


    print("Sent:", doc.doc_id)
```

4.5 How the key parts work

- **`Producer(...)`** creates the connection settings. Nothing is sent yet.
- **`json.dumps(...).encode("utf-8")`** turns the dictionary into a JSON string, then into bytes. Kafka messages are always bytes.
- **`produce()`** only places the message in a local queue; it returns immediately. The message is sent in the background.
- **`flush()`** blocks until the queue is empty. Calling it once **after** the loop is faster than calling it for every message.

> **Good to know:** `ir_datasets` returns `doc_id` as a **string** (for example `"1"`). The JSON therefore contains `"docID": "1"`. Flink's JSON format converts it to the `INT` column in the SQL tables.

4.6 Running it

```
python KafkaProducer.py
```

Python commands are case-sensitive on most systems, so use lowercase `python`.

5. Flink SQL: validating documents

5.1 Role of Flink

Flink is the **stream-processing and validation layer** between Kafka and OpenSearch. It processes documents as they arrive instead of waiting for the full dataset.

```
Cranfield dataset -> Kafka producer -> raw topic -> Flink SQL
                                                    |-> valid_docs   -> consumer ->
OpenSearch
                                                    |-> invalid_docs -> inspection
```

- **Consume** messages from the raw Kafka topic.
- **Deserialize** each JSON message into columns.
- **Validate** required fields against the rules below.
- **Route** each document to `valid_docs` or `invalid_docs`.

5.2 Technologies

Technology	Purpose
Apache Flink	Processes the stream of documents.
Apache Kafka	Provides the input stream and receives Flink's output.
Flink Kafka SQL connector	Lets Flink SQL read from and write to Kafka topics.

Technology	Purpose
Docker, Docker Compose, Dockerfile	Run the containers and build the Flink image with the connector.

5.3 Opening the Flink SQL client

```
docker exec -it flink-jobmanager ./bin/sql-client.sh
```

The container name comes from `container_name` in the Compose file. The Flink web UI is at http://localhost:8082 and shows running jobs.

> **Important:** Tables created in the SQL client live only in that session's in-memory catalog. They disappear when you close the client, so keep your `CREATE TABLE` statements in a `.sql` file. The Kafka **topics and data** are not lost.

5.4 The source table

A Flink table over a Kafka topic does not store data; it is a **schema on top of the topic**. The source table describes the raw messages the producer wrote:

```
CREATE TABLE Documents (
    docID  INT,
    title  STRING,
    text   STRING,
    author STRING,
    bib    STRING
) WITH (
    'connector' = 'kafka',
    'topic' = '',                       -- same as KAFKA_TOPIC in .env
    'properties.bootstrap.servers' = 'kafka:29092',
    'properties.group.id' = 'flink-cranfield-source',
    'scan.startup.mode' = 'earliest-offset',
    'format' = 'json'
);
```

> **Check:** Use the definition you actually created for `Documents`; the topic name above is only an example.

5.5 The valid_documents table

This is a **sink table**: the place Flink writes validated documents to. It is backed by the Kafka topic `valid_docs`.

```
CREATE TABLE valid_documents (
    docID INT,
    title STRING,
    text STRING,
    author STRING,
    bib STRING,
    processed_at TIMESTAMP_LTZ(3)
) WITH (
    'connector' = 'kafka',
    'topic' = 'valid_docs',
    'properties.bootstrap.servers' = 'kafka:29092',
    'properties.group.id' = 'flink-cranfield-validation',
    'scan.startup.mode' = 'earliest-offset',
    'format' = 'json'
);
```

Setting	Meaning
`connector = kafka`	Use the Kafka SQL connector (the JAR from Section 3).
`topic = valid_docs`	Kafka topic that stores the validated documents.
`properties.bootstrap.servers`	Kafka address **as seen from inside Docker**: `kafka:29092`.
`properties.group.id`	Consumer group name. Used only when Flink **reads** from this table.
`scan.startup.mode`	Where reading starts (`earliest-offset` = from the beginning). Also only used when **reading**, so it has no effect when this table is just a sink. It is harmless to keep.
`format = json`	Rows are written as JSON objects, one per message.
`TIMESTAMP_LTZ(3)`	A timestamp with local time zone and millisecond precision. `processed_at` records when Flink processed the document.

5.6 Validation rules and the INSERT

```
INSERT INTO valid_documents
SELECT
    docID,
    title,
    text,
    author,
    bib,
    CURRENT_TIMESTAMP AS processed_at
FROM Documents
WHERE docID IS NOT NULL
  AND docID > 0
  AND title IS NOT NULL
  AND TRIM(title) <> ''
  AND text IS NOT NULL
  AND TRIM(text) <> '';
```

A document is **valid** only if **all** of the following are true:

Rule	Why it matters
`docID IS NOT NULL AND docID > 0`	Every document needs a real positive identifier.
`title IS NOT NULL AND TRIM(title) <> ''`	The title must exist and must not be empty or only spaces. `TRIM` removes leading and trailing spaces, so `" "` is rejected.
`text IS NOT NULL AND TRIM(text) <> ''`	The body text is what we search, so empty documents are useless.

Two details worth understanding:

- `author` and `bib` are **not** validated, so a document without them is still valid.
- `INSERT INTO ... SELECT` in Flink starts a **continuous streaming job**. It keeps running and processes new messages as they arrive. You will see it as a running job in the web UI.
- In SQL, a comparison with `NULL` is neither true nor false. A row whose `WHERE` condition evaluates to NULL is dropped, which is why the `IS NOT NULL` checks come first.

5.7 The invalid_documents table

Rejected documents should not just disappear. We store them in a second sink so they can be inspected and fixed. A `reason` column explains why each one was rejected.

```
CREATE TABLE invalid_documents (
    docID INT,
    title STRING,
    text STRING,
    author STRING,
    bib STRING,
    reason STRING,
    processed_at TIMESTAMP_LTZ(3)
) WITH (
    'connector' = 'kafka',
    'topic' = 'invalid_docs',
    'properties.bootstrap.servers' = 'kafka:29092',
    'properties.group.id' = 'flink-cranfield-validation',
    'scan.startup.mode' = 'earliest-offset',
    'format' = 'json'
);

INSERT INTO invalid_documents

SELECT

    docID,

    title,

    text,

    author,

    bib,

    CASE

        WHEN docID IS NULL THEN 'Missing document ID'

        WHEN docID <= 0 THEN 'Document ID must be positive'

        WHEN title IS NULL OR TRIM(title) = ''

            THEN 'Missing title'

        WHEN text IS NULL OR TRIM(text) = ''

            THEN 'Missing document text'

        ELSE 'Invalid document'

    END AS validation_error,

    CURRENT_TIMESTAMP AS processed_at

FROM Documents

WHERE docID IS NULL

   OR docID <= 0

   OR title IS NULL

   OR TRIM(title) = ''

   OR text IS NULL
```

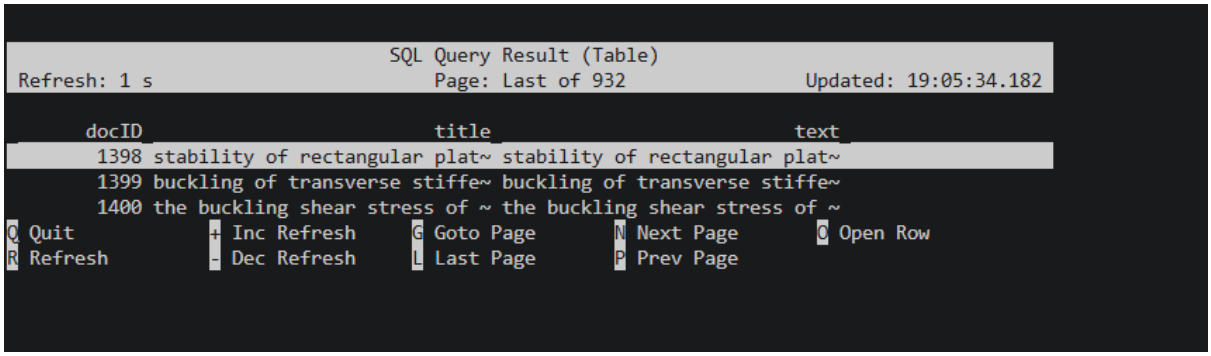
```
   OR TRIM(text) = '';
```

> **Why not just write NOT (valid condition)?** Because of NULL logic. If `title` is NULL, the valid condition evaluates to NULL and `NOT NULL` is still NULL, so the row would be dropped from **both** outputs. Writing each failure explicitly with `IS NULL OR ...` guarantees every document ends up in exactly one table.

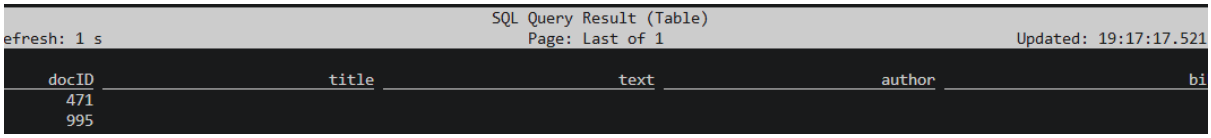
The `CASE` reports only the **first** rule a document fails. A document with several problems gets the first matching reason.

5.8 Running both inserts as one job

Valid Records:



Invalid Records:



5.9 Checking that it worked

- Open http://localhost:8082 and confirm the job is **RUNNING**.
- Read a few messages from the output topic:

```
docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server kafka:29092 --topic valid_docs --from-beginning --max-messages 3
```

- Do the same for `invalid_docs` to see which documents were rejected and why.
- Cranfield has about 1,400 documents. If fewer reach the index (see Section 7), the `invalid_docs` topic tells you which ones were filtered out.

> **Limit of SQL validation:** A message that is not valid JSON at all cannot be routed by these queries, because Flink fails to parse it before the SQL runs. For such cases the JSON format option `json.ignore-parse-errors` can skip bad records, but they are then dropped rather than stored.

5.10 How Flink fits into the pipeline

Component	Responsibility
Kafka producer	Reads Cranfield documents and publishes them to Kafka.
Kafka	Buffers and transports the messages.
Apache Flink	Consumes, validates and routes the documents.
Kafka consumer	Reads `valid_docs` and indexes documents in OpenSearch.
Amazon OpenSearch	Stores and searches the documents.
Search interface	Lets users find relevant papers.

6. Kafka consumer: loading OpenSearch

6.1 Introduction

The consumer reads the validated documents that Flink published to `valid_docs` and indexes each one in the `cranfield-documents` index of Amazon OpenSearch. It is the link between the streaming pipeline and the search system.

6.2 Responsibilities

- **Connect to Kafka** and subscribe to `valid_docs`.
- **Poll** for messages and check each one for Kafka errors.
- **Deserialize** the JSON bytes into a Python dictionary.
- **Connect to Amazon OpenSearch** using AWS-signed requests over HTTPS.
- **Index** each document into `cranfield-documents`.
- **Handle errors** and keep processing the following messages.

6.3 Libraries and installation

Library	Purpose
`confluent-kafka`	Consumes messages from Kafka.
`opensearch-py`	Connects to OpenSearch and indexes documents.
`boto3`	AWS SDK: finds the AWS credentials.
`python-dotenv`	Loads settings from the `.env` file.

```
python -m pip install confluent-kafka opensearch-py boto3 python-dotenv
```

`json` and `os` are part of Python and need no installation.

6.4 Configuration

```
KAFKA_BOOTSTRAP_SERVERS=localhost:9092
KAFKA_TOPIC=valid_docs
OPENSEARCH_HOST=your-domain-endpoint.us-east-1.es.amazonaws.com
AWS_REGION=us-east-1
```

- `OPENSEARCH_HOST` is the domain endpoint **without** `https://`.
- The consumer also needs AWS credentials whose IAM identity is allowed to access the domain (for example from `aws configure` or environment variables).
- The variable names must match the ones used in your Python code.

6.5 The complete script

```
import os

from opensearchpy import OpenSearch, RequestsHttpConnection, AWSV4SignerAuth

import boto3

from confluent_kafka import *

import json

from dotenv import load_dotenv


load_dotenv()


consumer_config= {

    "bootstrap.servers": os.getenv("CONSUMER_CONFIG"),

    "group.id": os.getenv("GROUP_AWSID"),

    "auto.offset.reset": "earliest",

    "enable.auto.commit": False

}

consumer= Consumer(consumer_config)


topic_aws= os.getenv("KAFKA_AWS_TOPIC")

index_name = os.getenv("OPENSEARCH_INDEX")


host = os.getenv("OPENSEARCH_HOST")

region = os.getenv("AWS_REGION")


credentials = boto3.Session().get_credentials()


auth = AWSV4SignerAuth(

    credentials,

    region,
```

```
    "es"
)

client = OpenSearch(
    hosts=[{"host": host, "port": 443}],
    http_auth=auth,
    use_ssl=True,
    verify_certs=True,
    connection_class=RequestsHttpConnection
)

if not client.indices.exists(index=index_name):
    client.indices.create(index=index_name)
    print("Created index:", index_name)

consumer.subscribe([topic_aws])
print("Waiting for Kafka messages...")

try:
    while True:
        msg = consumer.poll(1.0)

        if msg is None:
            continue

        if msg.error():
            if msg.error().code() == KafkaError._PARTITION_EOF:
                continue

            print("Kafka error:", msg.error())
            continue

        try:

            document = json.loads(msg.value().decode("utf-8"))
```

```
            if not isinstance(document, dict):
                raise ValueError("Expected a JSON object")



            document_id = document.get("id")


            if document_id is not None:
                client.index(
                    index=index_name,
                    id=str(document_id),
                    body=document,
                    refresh=False
                )
            else:
                client.index(
                    index=index_name,
                    body=document,
                    refresh=False
                )



            consumer.commit(message=msg, asynchronous=False)


            print("Indexed document:", document_id or "(generated ID)")


        except (json.JSONDecodeError, UnicodeDecodeError, ValueError) as e:
            print("Invalid Kafka message:", e)



except KeyboardInterrupt:
    print("Stopping consumer...")

finally:
    consumer.close()
    print("Kafka consumer closed.")
```

6.6 How it works

- **Signed connection:** `AWSV4SignerAuth` signs each request with your AWS credentials. The service name `"es"` is the identifier OpenSearch Service uses for signing.
- **`group.id`:** Kafka remembers how far this consumer group has read. Restarting the script continues where it stopped instead of starting again.
- **`auto.offset.reset = earliest`:** applies only when the group has no saved position yet.
- **`poll(1.0)`:** returns a message, or `None` after one second without one. Always check for `None` and `error()` before using the message.
- **`id=document["docID"]`:** without an explicit id OpenSearch generates a random one, so re-running the consumer would create duplicates. Using `docID` makes indexing **idempotent**: indexing the same document again overwrites it.
- **`consumer.close()`:** commits the final offsets and leaves the group cleanly.

6.7 Role in the architecture

Component	Responsibility
Kafka producer	Publishes raw Cranfield documents.
Apache Kafka	Buffers and transports messages.
Apache Flink	Validates documents and publishes valid records to `valid_docs`.
Kafka consumer	Reads `valid_docs` and sends documents to OpenSearch.
Amazon OpenSearch	Stores documents and provides full-text search.
Search interface	Lets users search for research papers.

7. Amazon OpenSearch: index, search and results

7.1 Creating the domain

I created an Amazon OpenSearch Service domain named `cranfield-search` in the AWS **US East (N. Virginia)** region. A domain is the managed OpenSearch cluster that stores and searches the documents. It was configured to use HTTPS and AWS authentication to secure access to the endpoint.

7.2 The index

An **index** is a collection of documents that OpenSearch organizes so their fields can be searched quickly. The documents are stored in an index named `cranfield-documents`. Each document has these fields:

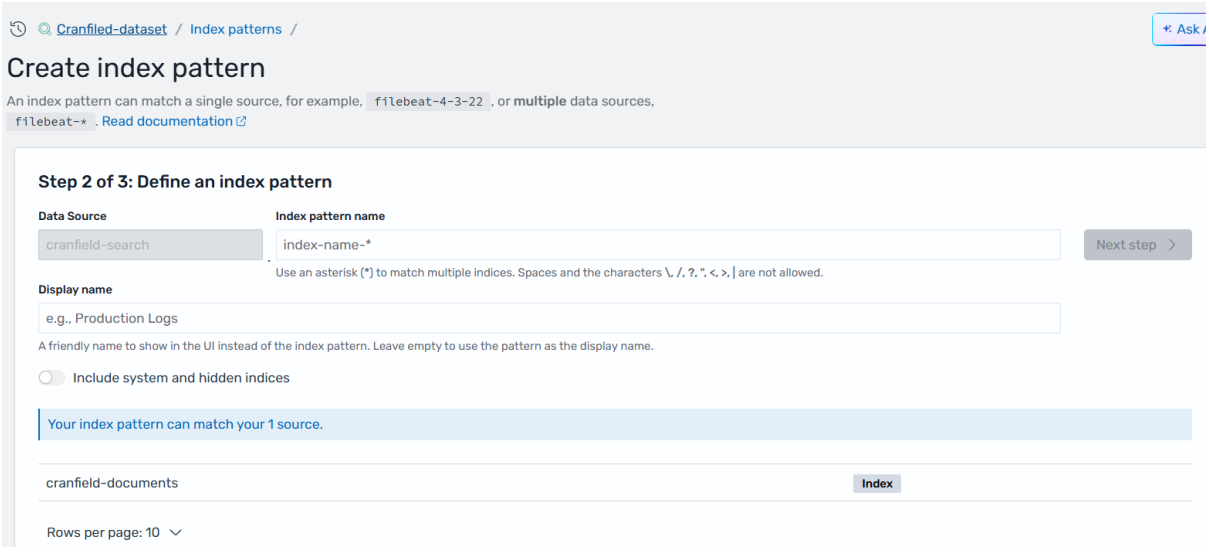
Field	Meaning
`docID`	The document's identifier.
`title`	The research paper's title.
`text`	The paper's text or abstract.
`author`	The author's name.

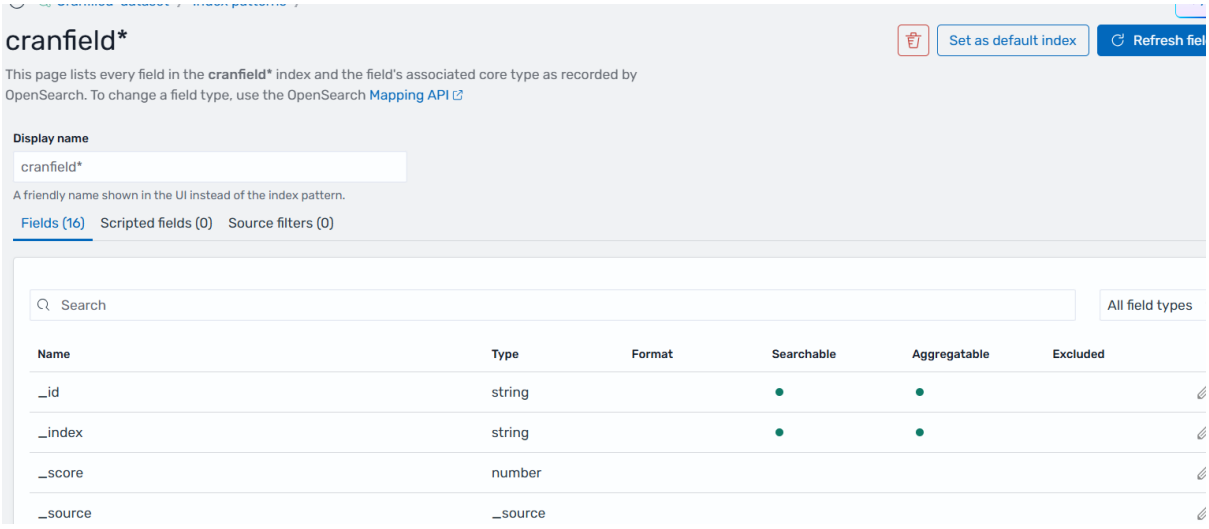
Field	Meaning
`bib`	Bibliographic information.
`processed_at`	The timestamp added by Flink during processing.

I checked the index and its field mappings to confirm the documents were stored with searchable fields.

7.3 The index pattern

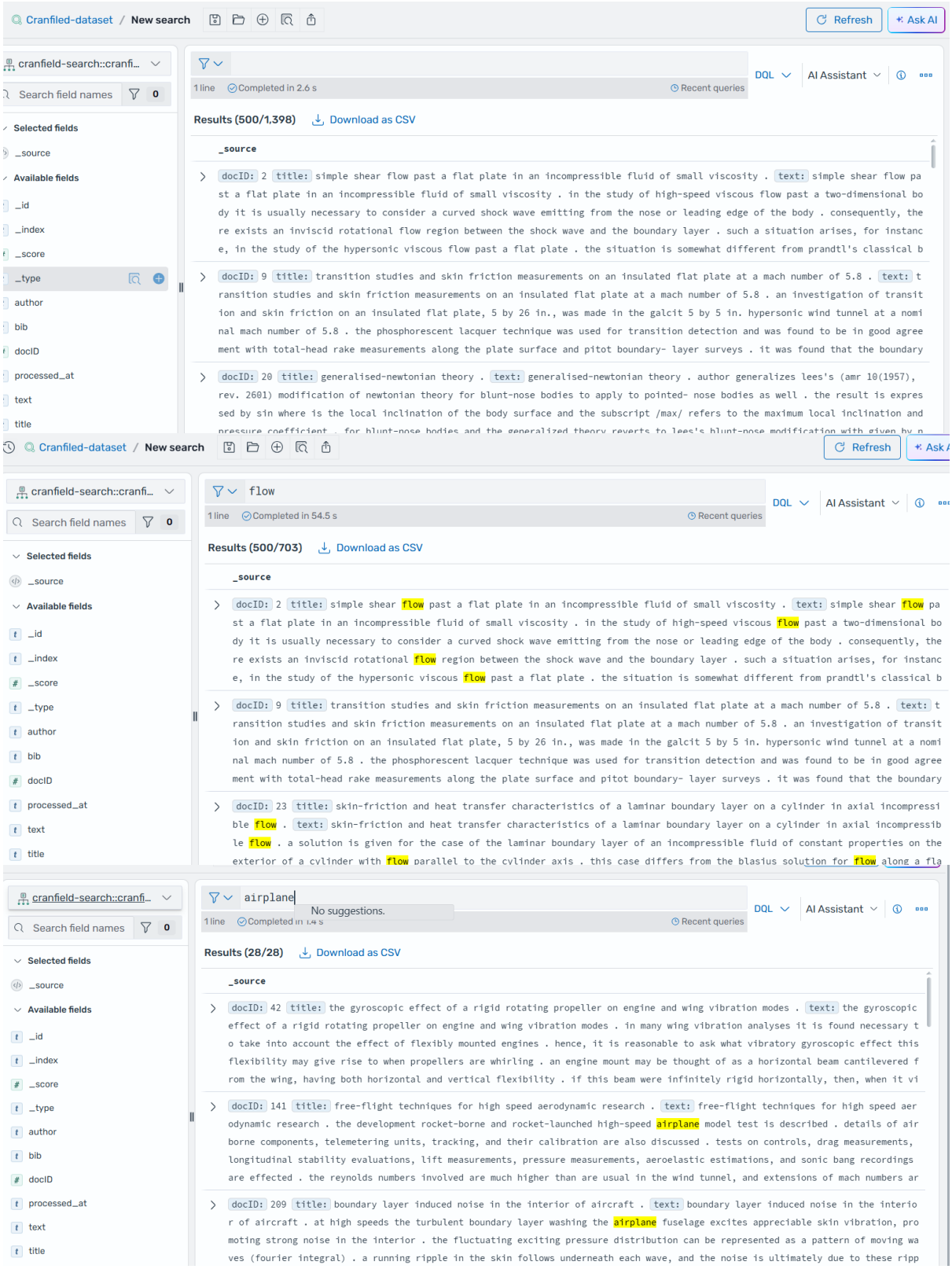
An **index pattern** (called a *data view* in some versions) tells OpenSearch Dashboards which index to display and explore. It does **not** copy or create documents; it only points at the existing index. I chose not to use a time filter, because the project searches papers by content, not by date.





7.4 Searching the documents

To test retrieval I searched for research topics, including `turbulent flow`, `aircraft aerodynamics` and `heat transfer`. The results came back **ranked by relevance**, with papers on heat transfer in turbulent shear flow appearing among the matches for `turbulent flow`.

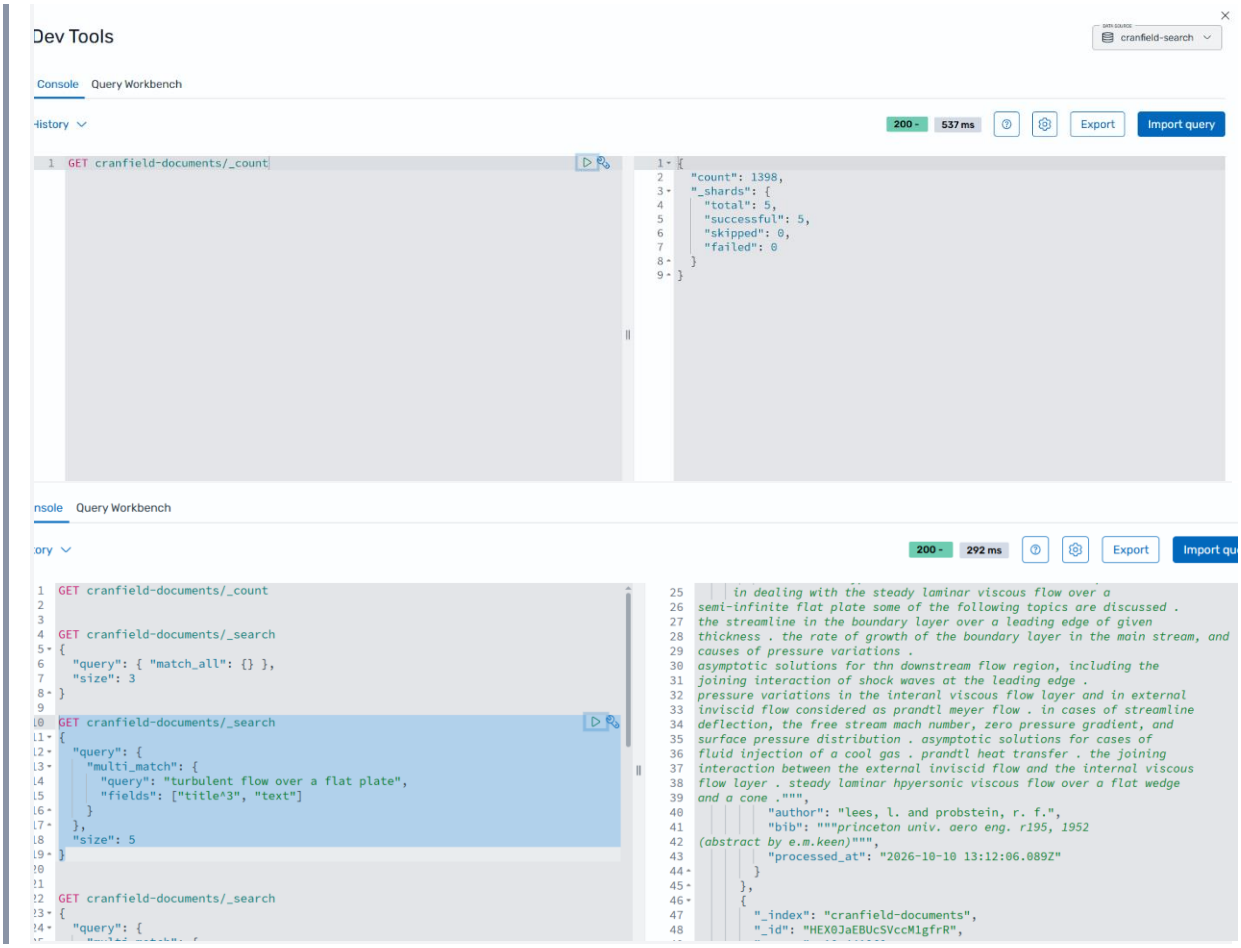


7.5 How OpenSearch search works

OpenSearch uses an **inverted index**: instead of storing "document -> words", it stores "word -> list of documents that contain it". At indexing time the text is split into lowercase terms (analysis). A query is analyzed the same way, the matching documents are looked up directly, and they are scored with the **BM25** relevance formula (rare terms and frequent matches within a document raise the score).

A multi-field query that gives titles more weight than body text:

```
{
  "query": {
    "multi_match": {
      "query": "turbulent flow",
      "fields": ["title^3", "text"]
    }
  },
  "size": 10
}
```



Part	Meaning
`multi_match`	Searches several fields with one query.
`title^3`	A match in the title counts three times as much as a match in the text.
`text`	Searches the main document text.
`size: 10`	Returns at most ten results.

> **Why so many matches?** By default a query with several words uses **OR**: a document matches if it contains `turbulent` **or** `flow`. That is why a broad query matches many documents. To require both words, add `"operator": "and"` to the `multi_match`; to require them next to each other, use `"type": "phrase"`.

7.6 Results and verification

During testing, the `cranfield-documents` index held approximately **1,398 documents**, and searching for `turbulent flow` returned approximately **744 matches**. This showed that the documents were indexed and full-text search was working.

> **Worth checking:** The Cranfield collection has about 1,400 documents, so the index holds slightly fewer than the original. If that gap comes from the Flink validation rules, those documents will be in `invalid_docs`, with the reason given. Counting that topic would confirm it and makes a good verification step to report.

8. Cleaning up AWS resources

After testing, I requested deletion of the `cranfield-search` domain to avoid ongoing AWS compute and storage charges. AWS accepted the request and the domain entered the **Deleting** state.

- Deleting the domain removes the hosted OpenSearch environment **and its indexed data** once the deletion completes.
- The local code, the Cranfield dataset and the Kafka and Flink containers are unaffected.
- To run the full pipeline again, create a new OpenSearch domain, update `OPENSEARCH_HOST`, and re-run the consumer (the Kafka topic still holds the validated documents).